

RRC-Raptor-V4H

- Porting Guide

V0.0.4

Contents

i.	VERSION LIST	4
ii.	HASHTAG	5
1.	Overview.....	6
2.	System Requirements.....	7
3.	Setup.....	8
3.1.	Host Setup	9
3.2.	Host Packages.....	10
3.3.	Yocto Project Setup	11
3.4.	Source Download	12
3.4.1.	Using Build Script	13
3.5.	File Deployment	14
4.	Install BSP	15
4.1.	Writing Boot Loaders.....	16
4.1.1.	DIP Switch Download mode.....	17
4.1.2.	Download IPL images	18
4.1.3.	Flash Writer	19
4.1.4.	DIP Switch Boot mode.....	22
4.2.	TFTP Server	23
4.3.	NFS Server	24
4.4.	Starting of U-Boot and Linux	25
5.	APPENDIX	28

LIST OF FIGURES

FIGURE 1. LAYOUT	15
FIGURE 2. BOOT DIPSWITCH	15
FIGURE 2. PUTTY SETTING.....	19

i. VERSION LIST

Description	Version	Date	Author
Porting Guide	V0.0.1	2024/09/12	kevinlin
Modify 3.5(Page14) 、 4.1(Page16~Page22) 、 4.4(Page25)	V0.0.2	2024/10/28	kevinlin
Modify 3.2(Page10) 、 4.1.3(Page19) , File attribute changes	V0.0.3	2024/12/19	kevinlin
Adjust BSP 3p28 hyperlink Cover range: - Section 3.4 - Section 4.1.2 - Section 5	V0.0.4	2026/03/06	Felix Hsu

ii. HASHTAG

#V4H

#u-boot

#kernel

#BSP

#ubuntu

1. Overview

Target :

Introduce how to download, compile and install RRC_Raptor files.

2. System Requirements

Reference PC software :

Distributor : Ubuntu
Description : Ubuntu 20.04.4 LTS
Release : 20.04
Codename : focal
Kernel : 5.15.0-43-generic

Reference PC hardware :

PC : Intel i9-11900K @ 2.50GHz
Width : 64 bits
Memory : 64G
Block device : 7.3TB Disk

3. Setup

Reference document :

Refer to <https://docs.yoctoproject.org/current/brief-yoctoprojectqs/index.html> for detail.

BB_VERSION	= 1.46.0
BUILD_SYS	= x86_64-linux
TARGET_SYS	= aarch64-poky-linux
MACHINE	= raptor
DISTRO	= poky
DISTRO_VERSION	= 3.1.11
SOC_FAMILY	= rcar:rcar-v4x:r8a779g0

META

meta-yocto-bsp	= tmp: 74b22db6879b388d700f61e08cb3f239cf940d18
meta-rcar-adas	= tmp: 89ca1415e3598789e9004363f9a38c1b3f41912d
meta-rtx	= v4h-raptor/meta-rtx:\${AUTOREV}
meta-networking	= tmp: 814eec96c2a29172da57a425a3609f8b6fcc6afe

3.1. Host Setup

Information :

When building on a machine running Ubuntu, the minimum hard disk space required is about 50 GB. It is recommended that at least 120 GB is provided, which is enough to compile all backends together.

The recommended minimum Ubuntu version is 20.04 or later.

The first download of source code and host setup must be executed when there is an Internet connection.

3.2. Host Packages

Essential Yocto Project host packages are :

```
$ sudo apt-get install gawk wget git-core diffstat unzip texinfo gcc-multilib  
build-essential chrpath socat cpio python python3 python3-pip python3-pexpect  
xz-utils debianutils iputils-ping python3-git python3-jinja2 libegl1-mesa  
libSDL1.2-dev pylint3 xterm libarchive-zip-perl rsync curl zstd lz4 libssl-dev
```

3.3. Yocto Project Setup

Git is set up properly with the commands below :

(this step may not be needed if the data has already been set)

```
$ git config --global user.name "Your Name"  
$ git config --global user.email "Your Email"  
$ git config --list
```

3.4. Source Download

Source Code :

```
$ mkdir RRC-Raptor-V4H  
$ cd RRC-Raptor-V4H  
$ git clone https://github.com/RetronixTechInc/rcar-bsp -b v4h-raptor  
$ git checkout -b bsp-3p28 raptor-bsp-3p28
```

3.4.1. Using Build Script

Execute the build script :

Note: If you have executed Yocto build before, remove your build directory (\$WORK/build-*) so that future builds start from a clean environment.

Retronix provides a script for users to simplify their building flow.

```
$ cd rcar-bsp  
$ ./build_yocto_v3p28.sh
```

3.5. File Deployment

u-boot 、 Kernel & BSP file :

After successfully compiling, the binaries will be created in
[build-raptor-adas/tmp/deploy/images/raptor/] directory with the below
description :

Image	← kernel file
r8a779g0-raptor.dtb	← kernel device tree
rcar-image-adas-v4h-\${DATE}.rootfs.tar.bz2	← file system
└─u-boot-elf-raptor.srec	← u-boot file

Each image build creates a U-Boot, a kernel, and an image type based on
the IMAGE_FSTYPES defined in the machine configuration file.

Most machine configurations provide a rootfs image (.tar.bz2).
The SD card image contains a partitioned image (with U-Boot, kernel, kernel
modules, rootfs, etc.) suitable for booting the corresponding hardware.

4. Install BSP

Layout :

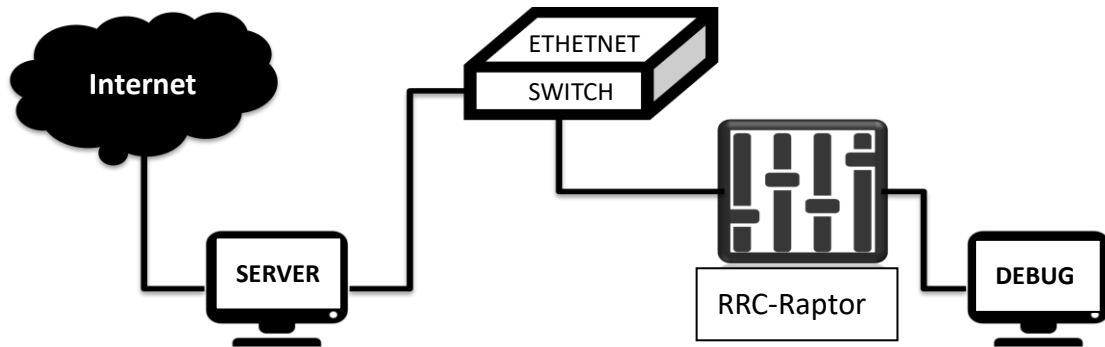


Figure 1. Layout

RRC-Raptor Kid DIP Switch :

SW26	SW35	SW31	SW27	SW30
1111 1111	1111 1111	1111 1111	1111 1111	1111 1111
SW28	SW36	SW33	SW29	SW32
0100 1011	0111 1110	1111 0100	1111 0111	1111 1111

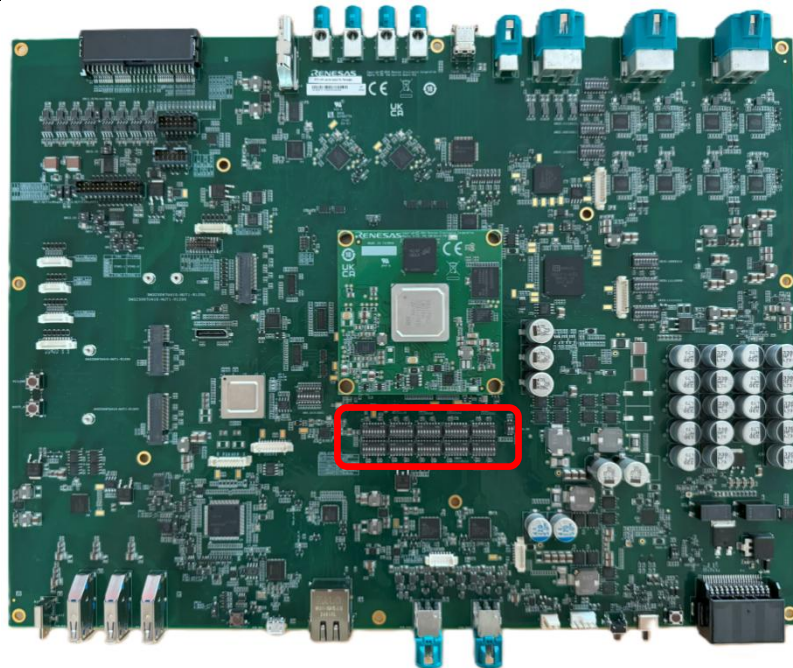


Figure 2. Boot dipswitch

4.1. Writing Boot Loaders

Introduces how to replace the u-boot file and how to write IPL.

4.1.1. DIP Switch Download mode

Download mode for RRC-Raptor device DIP switch :

SW26	SW35	SW31	SW27	SW30
1111 1111	1111 1111	1111 1111	1111 1111	1111 1111
SW28	SW36	SW33	SW29	SW32
0100 0000	0111 1110	1111 0100	1111 0111	1111 1111

4.1.2. Download IPL images

Reference <https://github.com/RetronixTechInc/rcar-bsp/tree/raptor-bsp-3p28>

Download IPL images.

[RRC-V4H-Raptor-IPL.zip](#)

[**Host PC server** in ubuntu system]

Extract RRC-V4H-Raptor-IPL.zip and get file with the below description :

	bl31.srec	
	bootparam_sa0.srec	
	cert_header_sa9.srec	
	cr52_loader.srec	
	dummy_fw.srec	
	dummy_rtos.srec	
	FlashWriter_V4H.sh	
	ICUMX_Flash_writer_SCIF_DUMMY_CERT_EB203000_V4H.mot	
	icumx_loader.srec	
	u-boot-elf.srec	← u-boot file
	u-boot_args.bin	

Update the boot file method and replace **u-boot-elf.srec** with the **u-boot-elf-raptor.srec** compiled in Chapter [3.5. File Deployment](#) . Other files remain unchanged.

4.1.3. Flash Writer

Connect USB Host connector of Ubuntu Host PC that is virtual COM port to Carried Board(CN23) with USB cable for displaying console

[**Host PC server** in ubuntu system]

Activate the Terminal Software PuTTY on Ubuntu Host PC. Configure the PuTTY on Ubuntu Host PC as followings:

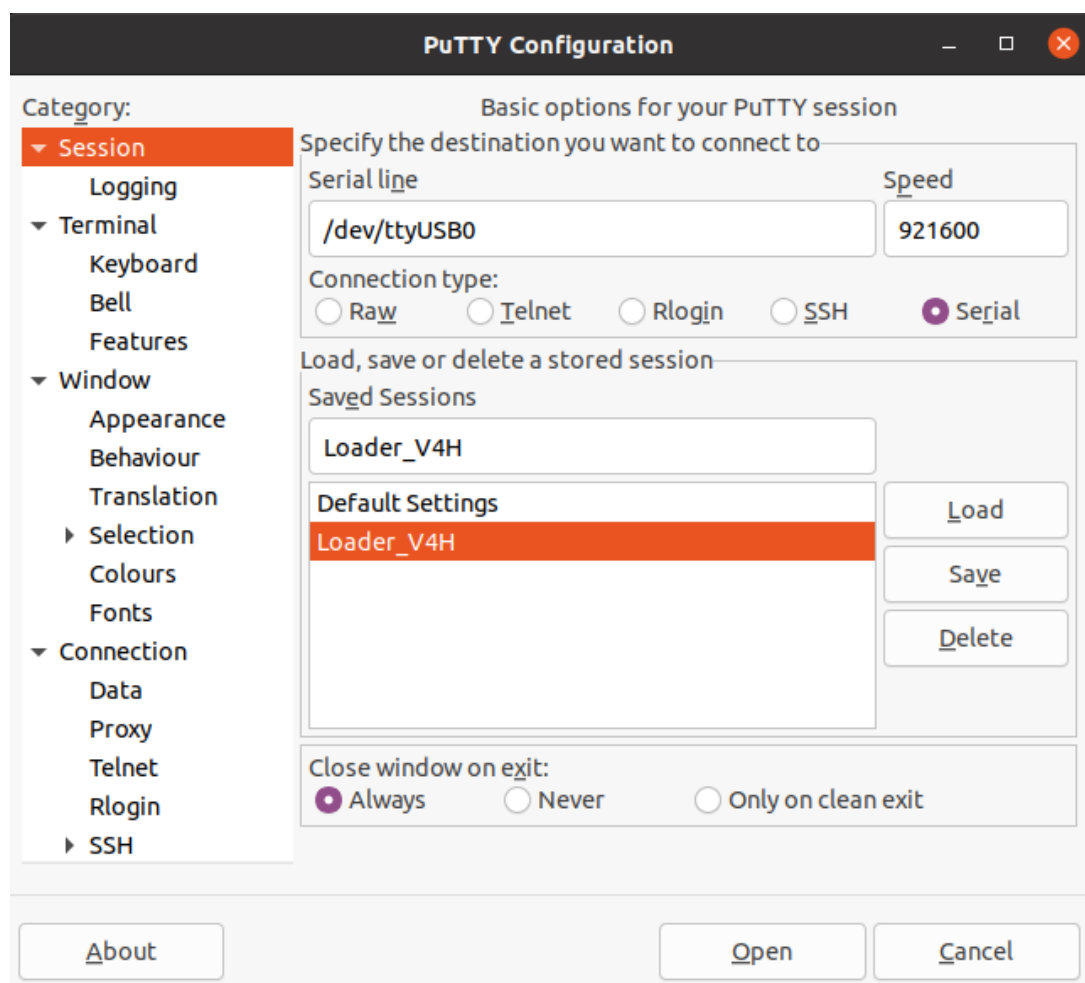


Figure 3. PuTTY setting

Execute “FlashWriter_V4H.sh” download firmware to QSPI Flash and eMMC

```
$ cd RRC-V4H-Raptor-IPL
$ sudo ./FlashWriter_V4H.sh -a 1
```

Flash writer log :

```
Updating all files to flash and eMMC.....
Load FlashWriter
/RRC-V4H-Raptor-IPL/ICUMX_Flash_writer_SCIF_DUMMY_CERT_EB203000_V4H.mo
t.....
sudo dd
if=/RRC-V4H-Raptor-IPL/ICUMX_Flash_writer_SCIF_DUMMY_CERT_EB203000_V4H.
mot of=/dev/ttyUSB0
490+1 records in
490+1 records out
251144 bytes (251 kB, 245 KiB) copied, 2.63326 s, 95.4 kB/s
OK
PARTITION_INFO = EB200000:000000:/RRC-V4H-Raptor-IPL/bootparam_sa0.srec
ARG_TOP_ADDR = EB200000
ARG_SAVE_ADDR = 000000
ARG_FILE_NAME = /RRC-V4H-Raptor-IPL/bootparam_sa0.srec
77+1 records in
77+1 records out
39540 bytes (40 kB, 39 KiB) copied, 0.340135 s, 116 kB/s
PARTITION_INFO = EB210000:040000:/RRC-V4H-Raptor-IPL/icumx_loader.srec
ARG_TOP_ADDR = EB210000
ARG_SAVE_ADDR = 040000
ARG_FILE_NAME = /RRC-V4H-Raptor-IPL/icumx_loader.srec
420+1 records in
420+1 records out
215386 bytes (215 kB, 210 KiB) copied, 2.24549 s, 95.9 kB/s
PARTITION_INFO = EB230000:240000:/RRC-V4H-Raptor-IPL/cert_header_sa9.srec
ARG_TOP_ADDR = EB230000
ARG_SAVE_ADDR = 240000
ARG_FILE_NAME = /RRC-V4H-Raptor-IPL/cert_header_sa9.srec
294+1 records in
294+1 records out
150604 bytes (151 kB, 147 KiB) copied, 1.54371 s, 97.6 kB/s
PARTITION_INFO = EB240000:280000:/RRC-V4H-Raptor-IPL/dummy_fw.srec
ARG_TOP_ADDR = EB240000
ARG_SAVE_ADDR = 280000
ARG_FILE_NAME = /RRC-V4H-Raptor-IPL/dummy_fw.srec
```

```
74+1 records in
74+1 records out
38258 bytes (38 kB, 37 KiB) copied, 0.326375 s, 117 kB/s
PARTITION_INFO = E6300000:480000:/RRC-V4H-Raptor-IPL/cr52_loader.srec
ARG_TOP_ADDR = E6300000
ARG_SAVE_ADDR = 480000
ARG_FILE_NAME = /RRC-V4H-Raptor-IPL/cr52_loader.srec
553+1 records in
553+1 records out
283144 bytes (283 kB, 277 KiB) copied, 2.9805 s, 95.0 kB/s
write finished!!!
PARTITION_INFO = E2100000:0D00:/RRC-V4H-Raptor-IPL/dummy_rtos.srec
ARG_TOP_ADDR = E2100000
ARG_SAVE_ADDR = 0D00
ARG_FILE_NAME = /RRC-V4H-Raptor-IPL/dummy_rtos.srec
288+1 records in
288+1 records out
147514 bytes (148 kB, 144 KiB) copied, 1.51053 s, 97.7 kB/s
PARTITION_INFO = 46400000:8D00:/RRC-V4H-Raptor-IPL/bl31.srec
ARG_TOP_ADDR = 46400000
ARG_SAVE_ADDR = 8D00
ARG_FILE_NAME = /RRC-V4H-Raptor-IPL/bl31.srec
221+1 records in
221+1 records out
113262 bytes (113 kB, 111 KiB) copied, 1.13905 s, 99.4 kB/s
PARTITION_INFO = 50000000:9500:/RRC-V4H-Raptor-IPL/u-boot-elf.srec
ARG_TOP_ADDR = 50000000
ARG_SAVE_ADDR = 9500
ARG_FILE_NAME = /RRC-V4H-Raptor-IPL/u-boot-elf.srec
3838+1 records in
3838+1 records out
1965402 bytes (2.0 MB, 1.9 MiB) copied, 21.2042 s, 92.7 kB/s
write finished!!!
$
```

When the terminal displays “**write finished!!!**”, the firmware download is completed.

4.1.4. DIP Switch Boot mode

Boot mode for RRC-Raptor device DIP Switch :

SW26	SW35	SW31	SW27	SW30
1111 1111	1111 1111	1111 1111	1111 1111	1111 1111
SW28	SW36	SW33	SW29	SW32
0100 1011	0111 1110	1111 0100	1111 0111	1111 1111

4.2. TFTP Server

BSP space :

[**Host PC server** in ubuntu system]

Make sure you are connected to the internet. If there is any error, try apt update -y to update your application repository database.

SETP 1 : Install tftpd-hpa.

```
$ sudo apt install tftpd-hpa -y
```

STEP 2 : Change the following line to give R/W permission to TFTP folder.

```
$ sudo vi /etc/default/tftpd-hpa
```

```
# /etc/default/tftpd-hpa

TFTP_USERNAME="tftp"
TFTP_DIRECTORY="/var/lib/tftpboot"
TFTP_ADDRESS=":69"
#TFTP_OPTIONS="--secure"
TFTP_OPTIONS="--secure --create"
```

SETP 3 : Restart the tftpd-hpa service.

```
$ sudo service tftpd-hpa restart
```

4.3. NFS Server

KERNEL & DTS space :

[**Host PC server** in ubuntu system]

STEP 1 : Install the NFS kernel server package and rpcbind.

```
$ sudo apt install nfs-kernel-server  
$ sudo apt install rpcbind
```

STEP 2 : Configuring NFS Server.

```
$ sudo vi /etc/exports
```

```
/export/rfs *(rw,async,no_root_squash)
```

STEP 3 : Notify the NFS server after making any changes to the export file

```
$ exportfs -a
```

STEP 4 : Restart the NFS Service.

```
$ sudo service nfs-kernel-server restart
```

4.4. Starting of U-Boot and Linux

[**Host PC server** in ubuntu system]

SETP 1 : Prepare filesystem and kernel image and dtb.

Store Kernel image and device tree to TFTP root directory.

```
$ mkdir -p /var/lib/tftpboot/root/  
$ cp Image /var/lib/tftpboot/root/  
$ cp r8a779g0-raptor.dtb /var/lib/tftpboot
```

Store filesystem rcar-image-adas-v4h-\${DATE}.rootfs.tar.bz2 to root directory of NFS server, and extract it as follows.

```
$ cp rcar-image-adas-v4h-${DATE}.rootfs.tar.bz2 /export/rfs  
$ cd /export/rfs  
$ tar xvf rcar-image-adas-v4h-${DATE}.rootfs.tar.bz2
```

[**RRC-Raptor Device** in Linux system]

SETP 1 : Boot on and stop autoboot.

Power on the RRC-Raptor Device. When the system executes the uboot image, the console screen displays "Hit any key to stop autoboot: *". Hit any key to enter command mode ;

```
Hit any key to stop autoboot: 0  
=>
```

SETP 2 : Reset **DEFAULT** environment variables.

```
=> env default -a
```

DEFAULT environment variables :

```
bootdelay=2  
cma_size=900M
```

```
ipaddr=192.168.0.20
serverip=192.168.0.1

loadaddr=0x48080000
loadaddr_dtb=0x48000000
loadaddr_dtbo=0x48060000

serverfold=/export/rfs
file_kernel=/boot/Image
file_dtb=/boot/r8a779g0-raptor.dtb
dtbo_pcie=/boot/r8a779g0-raptor-overlay-pcie1x4.dtb

pcie_check=if gpio input 0; then echo pcie2x2; else echo pcie1x4;run pcie1x4;fi
pcie1x4=fdt addr ${loadaddr_dtb};fdt resize 8192;${loadcmd} ${loadaddr_dtbo}
${dtbo_pcie};fdt apply ${loadaddr_dtbo}

load_kernel=${loadcmd} ${loadaddr} ${file_kernel};${loadcmd} ${loadaddr_dtb}
${file_dtb}

bootargs_nfs=setenv bootargs rw root=/dev/nfs nfsroot=${serverip}:${serverfold},
nfsvers=3 ip=dhcp cma=${cma_size},clk_ignore_unused

load_tftp=setenv loadcmd tftp

bootcmd_nfs=run bootargs_nfs;run load_tftp;run load_kernel;run pcie_check;booti
${loadaddr} - ${loadaddr_dtb}

bootargs_mmc=setenv bootargs rw root=/dev/mmcblk0p2 rootfstype=ext4
rootwait cma=${cma_size},clk_ignore_unused

load_emmc=setenv loadcmd ext4load mmc 0:2

bootcmd_emmc=run bootargs_mmc;run load_emmc;run load_kernel;run
pcie_check;booti ${loadaddr} - ${loadaddr_dtb}

bootcmd=run bootcmd_emmc
```

STEP 3 : Set U-Boot environment variables.

The command indicates to mount the root file system through the **NFS server with read/write options, the IP is 192.168.0.1** (default sample), the root directory It is /export/rfs, and the **client IP is 192.168.0.20**(default sample).(IP value depends on domain settings.)

xx:xx:xx:xx:xx:xx is the AVB0 port MAC address.

```
=> setenv -f ethaddr xx:xx:xx:xx:xx:xx
=> setenv ipaddr xx.xx.xx.xx
=> setenv serverip xx.xx.xx.xx
```

STEP 4 : Change the bootargs by U-Boot.

The default bootcmd is run bootcmd_emmc, so it needs to be changed to run bootcmd_nfs

```
=> setenv bootcmd ' run bootcmd_nfs '
```

STEP 5 : Save environment variables to persistent storage.

By the "saveenv" command, we do not need to execute Step 2 - Step 4 in next times.

```
=> saveenv
```

STEP 6 : Start Linux.

After board reset or execute the reset command, U-Boot is started. After countdown, Linux images are loaded, and boot messages are displayed.

```
=> reset
```

5. APPENDIX

RETRONIX Source Code Link :

<https://github.com/RetronixTechInc/rcar-bsp/tree/raptor-bsp-3p28>

README.md at v4h-raptor :

[README.md](#)

Download IPL images :

[RRC-V4H-Raptor-IPL.zip](#)